

PALANTIR FOUNDRY PROJECT ENABLEMENT PROGRAM

Phase 1 — Day 1

Project Architecture & Solution Walkthrough

2 Hours • 10-Day Foundation Accelerator

Presented by Rajesh Pasham

Where Today Fits In

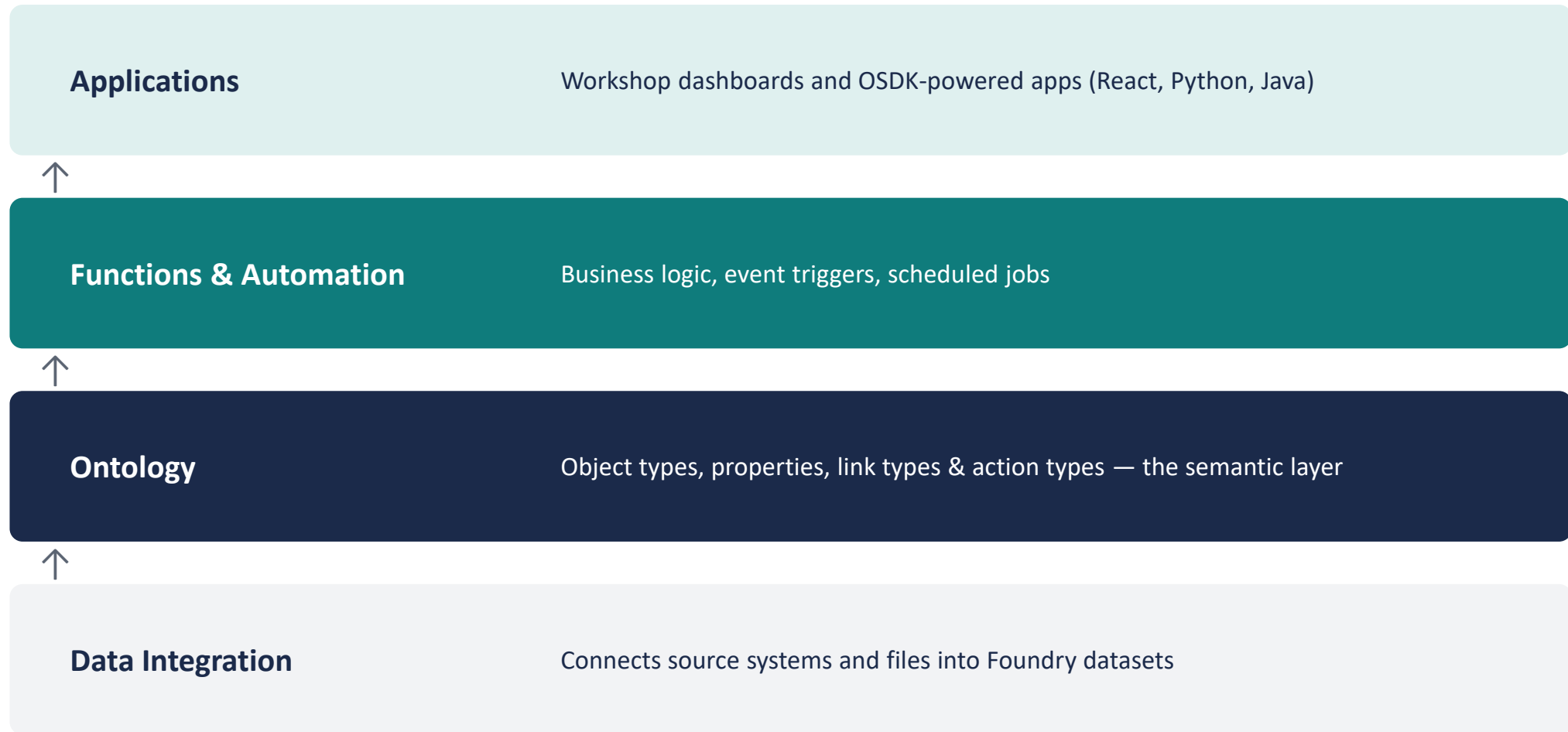


Day 1 of Phase 1 sets the shared architectural foundation the whole engagement builds on.

By the End of Today, You Will Be Able To

- Describe the major architectural layers of the Foundry platform
- Explain the Ontology's core building blocks: object types, properties, link types, and action types
- Navigate the Ontology Manager and locate key resources
- Describe how Workshop, OSDK applications, and Functions consume the Ontology
- Create your first Ontology object types in your own Developer Tier account
- Understand the coding standards and sprint structure for the rest of the engagement

The Foundry Platform, Built Bottom-Up



Each layer is built on top of the one below it — start with data, then model it, then automate it, then expose it.

Data Integration

- The entry point for any source system, file, or database into Foundry
- Data Connection sources bring data in on a schedule or via streaming
- Pipeline Builder (low-code) or code-based transforms (Python/SQL) clean and reshape it

For this project: UPS and Neology feeds will land here before reaching the Ontology

Functions & Automation

- Where business logic lives — validation rules, calculations, event handlers
- Functions back Action Types: every Ontology write ultimately runs through a Function
- Also power scheduled jobs (polling) and event-driven triggers

Later weeks: outbound API calls, scheduled polling, and notifications all live here

Applications

- The layer people and other systems actually interact with
- Workshop: low-code, drag-and-drop operational apps bound to the Ontology
- OSDK applications: fully custom React, Python, or Java apps for bespoke experiences

This engagement builds both: Workshop screens in Phase 2, and a React PWA starting Day 3

What Is the Ontology?

- A categorization of your organization's world — Palantir describes it as a digital twin
- Maps real datasets and models onto object types, properties, link types, and action types
- Object Type

Defines an entity or event — e.g. Vehicle, Transponder

- Property

A characteristic of an object type — e.g. status, make, year

- Link Type

A relationship between two object types — e.g. Vehicle ↔ Transponder

- Action Type

Defines how an object can be created or modified

A Familiar Analogy

A Dataset...

- A dataset is like a spreadsheet
- A row is one record
- A column is one field

...and an Object Type

- An object type is like a dataset
- An object is one instance (one row)
- A property is like one column

Ontology Manager Walkthrough

- Open it directly: add /workspace/ontology to your Foundry home page URL
- Top bar

Search for resources, create new ones, navigate or create branches

- Sidebar

Navigate between resources, pages, and applications

- Discover view

A customizable landing page highlighting recently modified and prominent object types

Object Type View & Property Editor

- Selecting an object type opens its detail view, including an Overview page listing properties
- Selecting a property opens the property editor: data type, display name, and constraints
- The same view shows an object type's Links tab and (once defined) its Actions tab

You will spend most of today's hands-on exercise in this exact view

Object Type: Vehicle

Property	Type	Notes
vehicleId	String	Primary key / title
vin	String	17-character VIN
make / model	String	e.g. Ford, Transit-350
year	Integer	
licensePlate	String	
status	String	Active / Maintenance / Inactive
currentLocation	String	Facility name

Object Type: Transponder

Property	Type	Notes
transponderId	String	Primary key / title
serialNumber	String	
status	String	Active / In Stock / Faulty
issueDate	Date	ISO format
deviceType	String	e.g. RFID-Gen2, BLE-Beacon

Link Type: Vehicle ↔ Transponder



- Cardinality: one-to-one for Day 1 (a vehicle has at most one active transponder)
- Foreign key: `Transponder.assignedVehicleId` → `Vehicle.vehicleId`
- Later sprints extend this to one-to-many for full lifecycle history

Your Data Model at a Glance

Vehicle

- vehicleId (PK)
- vin
- make / model
- year
- licensePlate
- status
- currentLocation

assignedTransponder (1:1)

Transponder

- transponderId (PK)
- serialNumber
- status
- issueDate
- deviceType
- assignedVehicleId (FK)

Action Types — A Quick Preview

- Action types define how an object can be created, updated, or deleted
- Every write to the Ontology goes through an action — never a direct database write
- Actions can be triggered from Workshop buttons, APIs, or Functions

We build our first custom action types in Week 2 (Advanced Ontology Modeling)

Mutability & Governance — A Preview

- Foundry objects support append and update patterns — direct deletes are avoided in operational systems
- Version history and audit trails are built on this same append/update model

Week 3 (Mutable Operational Applications) covers this in depth, with a full Inspection Workflow lab

- Governance is enforced at every layer: object-level permissions, action-level permissions, and branch-based change review

Workshop — Operational Applications on the Ontology

- Workshop is Foundry's no-code / low-code application builder
- Widgets (tables, forms, maps, charts) bind directly to Ontology objects and actions
- Used for internal, operational tools — dashboards, review queues, data entry

In this engagement: Inventory, Shipment, and Check-in/Check-out screens are built in Workshop

OSDK, Developer Console & Foundry CLI

- Ontology SDK (OSDK)

A generated, typed client library (TypeScript, Python, Java) for reading and writing Ontology objects from code

- Developer Console

Where OSDK applications and OAuth clients are created and managed

- Foundry CLI & Code Repositories

Local development, version control, and publishing pipelines for Functions and applications

- SuperRepo (newest capability)

Define Ontology, Functions, and frontend together in one repository, with the OSDK generated automatically

React + OSDK Architecture Preview



Modules 2–3 (Days 3–6) build this end to end, starting with your first OSDK application on Day 2.

Security — A Preview

- Role-Based Access Control (RBAC) restricts who can see or act on which object types
- Attribute-Based Access Control (ABAC) restricts access based on data values — e.g. a manager's assigned region

Week 10 (Enterprise Security) builds a Regional Operations Dashboard enforcing both

- For now: know that every object and action you build already has a permissions model attached, even before you configure it explicitly

Coding Standards for This Engagement

- Object & property naming: lowerCamelCase for properties, PascalCase for object type display names
- TypeScript for all React/OSDK application code; Python for Functions and data transforms
- One Code Repository per application; Ontology changes reviewed before publishing
- Every Pull Request references its Sprint and includes a short description of the Ontology or UI change
- Commit early, commit often — small, reviewable changes over large batch commits

Phase 2 Sprint Plan — Recap

Sprint	Focus
Sprint 1	Ontology — Shipment, Inventory, Transponder, Bin
Sprint 2	React App — Add / Remove Transponder
Sprint 3	Shipment APIs — External integrations, polling
Sprint 4	Barcode, Inventory & Geospatial Visualization
Sprint 5	Testing, Deployment, Security

Everything you build in Phase 1 becomes the foundation for these sprints.

Foundry Glossary Cheat-Sheet

- Ontology

The semantic layer mapping data to object types, properties, links, actions

- Object / Property / Link / Action Type

The four core Ontology building blocks

- Ontology Manager

The UI for building and browsing the Ontology

- OSDK

Generated, typed client library for code access to the Ontology

- Workshop

Low-code application builder for operational tools

Day 1 FAQ

- “Do I need coding experience for today?”

No — today is UI-driven in Ontology Manager. Coding starts Day 2.

- “What if my Developer Tier sign-up isn't available in my region?”

Use the engagement-provisioned Foundry environment instead — every step works identically.

- “Can I use different sample data than the provided vehicles/transponders?”

Yes, but the hands-on steps and troubleshooting notes assume the provided datasets.

What You'll Build in the Next 40 Minutes

- Sign in to your own Foundry Developer Tier account
- Create the Vehicle object type with 8 properties
- Create the Transponder object type with 6 properties
- Link them with a one-to-one relationship
- Upload the provided sample datasets and back your object types with real data

Full step-by-step instructions are in today's Hands-On Exercise document

Today's Definition of Done

- Vehicle object type published, with all 8 properties, backed by real data
- Transponder object type published, with all 6 properties, backed by real data
- assignedTransponder link correctly connects at least 10 Vehicle/Transponder pairs
- You can browse both object types and their linked objects in Ontology Manager

(Stretch) A simple Workshop table view displays your Vehicle objects

Questions?

Open floor — architecture, tooling, or anything from today's walkthrough.

Next Session — Day 2

Module 1: OSDK Architecture

- OSDK overview and SDK concepts
- Architecture and Object APIs
- Actions and Queries
- Authentication
- *Hands-on lab: develop your first OSDK application*